```
template<typename T>
class lock_free_stack
{
private:
    struct node
    {
        std::shared_ptr<T> data;
        node* next;

        node(T const& data_):
            data(std::make_shared<T>(data_))
        {}
    };

    std::atomic<node*> head;
public:
    void push(T const& data)
    {
        node* const new_node=new node(data);
        new_node->next=head.load();
        while(!head.compare_exchange_weak(new_node->next,new_node));
    }
    std::shared_ptr<T> pop()
    {
        node* old_head=head.load();
        while(old_head &&
            !head.compare_exchange_weak(old_head,old_head->next));
        return old_head ? old_head->data : std::shared_ptr<T>();
    }
};
```

❶ Data is now held by pointer

❷ Create std::shared_ptr for newly allocated T

❸ Check old_head is not a null pointer before you dereference it

❹

The data is held by the pointer now ❶, so you have to allocate the data on the heap in the node constructor ❷. You also have to check for a null pointer before you dereference `old_head` in the `compare_exchange_weak()` loop ❸. Finally, you either return the data associated with your node, if there is one, or a null pointer if not ❹. Note that although this is *lock-free*, it's not *wait-free*, because the `while` loops in both `push()` and `pop()` could in theory loop forever if the `compare_exchange_weak()` keeps failing.

If you have a garbage collector picking up after you (like in managed languages such as C# or Java), you're finished; the old node will be collected and recycled once it's no longer being accessed by any threads. However, not many C++ compilers ship with a garbage collector, so you generally have to tidy up after yourself.